

Práctica con Jenkins

De "no sé qué es Jenkins" a un pipeline DevSecOps completo, funcionando y con resultados en DefectDojo.

Jenkins

Pipeline as Code

Gitleaks

SonarQube

Trivy

OWASP ZAP

DefectDojo

Tres niveles, cada uno con resultado visible. Si te atascas en uno, el siguiente sigue teniendo sentido. Todo lo que necesitas está en este documento.

1. Qué vas a hacer hoy

En la sesión anterior montaste un pipeline de seguridad en **GitLab CI**: hacías `git push` y, solo, se ejecutaban siete controles de seguridad. Hoy vas a montar **exactamente lo mismo** con otro orquestador: **Jenkins**.

¿Por qué repetirlo con otra herramienta? Porque lo que de verdad aprendiste no fue GitLab, fueron los **conceptos**: etapas, artefactos, credenciales, quality gates. Al hacerlo en Jenkins vas a comprobar que esos conceptos viajan contigo. Y porque Jenkins es, con diferencia, el CI/CD que más te vas a encontrar en empresas grandes y en entornos aislados de internet.

Jenkins en cinco líneas

- **Qué es:** un servidor de automatización. Ejecuta tareas (*jobs*) cuando tú (o un evento) se lo pides.
- **Qué NO es:** no es un escáner de seguridad, ni un repositorio de código. Es **quien llama** a las herramientas.
- **Cómo se le dan órdenes:** con un fichero de texto llamado `Jenkinsfile` que vive en tu repositorio.
- **Dónde ejecuta:** en *agentes*. En este laboratorio, en el propio servidor Jenkins.
- **Cómo ejecuta herramientas que no tiene instaladas:** lanzando **contenedores Docker**.

El pipeline que vas a construir

1 · Secretos GitLeaks	2 · SAST SonarQube	3 · SCA Trivy fs	4 · Build Docker	5 · Imagen Trivy image	6 · DAST OWASP ZAP	7 · Informe DefectDojo
--------------------------	-----------------------	---------------------	---------------------	---------------------------	-----------------------	---------------------------

Los tres niveles de la práctica

Nivel	Tiempo	Qué consigues
Nivel 1	~10 min	Tu primer job funcionando. Entiendes la interfaz, el log y el ciclo build → resultado.
Nivel 2	~15 min	Traes código real desde GitLab usando credenciales y ejecutas un escáner de seguridad de verdad.
Nivel 3	~15 min	El pipeline completo, leído desde el repositorio, con resultados en SonarQube y DefectDojo.
Reto	~5 min	Conviertes un aviso en un bloqueo : el quality gate.

2. Accesos y reglas del laboratorio

Herramienta	URL (tu navegador)	Usuario / contraseña
Jenkins	<code>http://localhost:8180</code>	el que te dé el instructor
GitLab	<code>http://localhost:8929</code>	<code>root</code> / <code>DevSecOps2024!</code>
SonarQube (puede estar apagado)	<code>http://localhost:9000</code>	<code>admin</code> / <code>DevSecOps2024!</code>
DefectDojo	<code>http://localhost:8080</code>	<code>admin</code> / <code>DevSecOps2024!</code>
Juice Shop	<code>http://localhost:3000</code>	(es el objetivo del DAST)

Si trabajas contra el servidor del aula y no en tu propia máquina, sustituye `localhost` por la IP que te indique el instructor.

Las dos direcciones de cada servicio. Tú entras a las herramientas por `localhost:PUERTO`. Pero el pipeline se ejecuta **dentro de contenedores**, y desde ahí `localhost` es el propio contenedor, no tu máquina. Por eso, **dentro del pipeline** siempre se usa el **nombre del contenedor**:

Servicio	URL desde el pipeline
GitLab	<code>http://devsecops-gitlab:8929</code>
SonarQube	<code>http://devsecops-sonarqube:9000</code>
DefectDojo (API)	<code>http://devsecops-defectdojo:8081</code>
Juice Shop (objetivo DAST)	<code>http://devsecops-juiceshop:3000</code>

Si algún día un job te dice *"connection refused"*, lo primero que debes mirar es esto.

SonarQube puede estar apagado. El laboratorio va justo de memoria y, con toda la clase lanzando pipelines a la vez, el instructor suele apagarlo. Si es el caso, en el log verás `SonarQube no responde → etapa SAST OMITIDA` y la etapa saldrá en **verde: no es un fallo tuyo**, y las otras seis etapas funcionan igual. El instructor lo encenderá al final de la clase para enseñar los resultados.

Jenkins es compartido con el resto de la clase. Para no pisaros:

- Pon **tu nombre** en todo lo que crees: job `pipeline-ana`, credencial `gitlab-cred-ana` ...
- **No borres** jobs ni credenciales que no sean tuyos.
- Si tu build se queda en cola (*pending*), es que otro compañero está ejecutando. Espera: es normal.

3. Primer contacto: la interfaz de Jenkins

Entra en `http://localhost:8180` y haz login. Verás el **Dashboard**. Antes de tocar nada, localiza estas cinco zonas:

Dónde	Qué es
+ New Item (arriba a la izquierda)	Crear un job nuevo. Aquí empezarás en un minuto.
Lista central	Todos los jobs, con una bola de color: azul/verde = bien, amarillo = con avisos, rojo = falló. El icono del sol/nube es la "meteorología": la tendencia de los últimos builds.
Build Executor Status (barra izquierda)	Cuántas ejecuciones caben a la vez y cuáles están corriendo ahora mismo.
Manage Jenkins	Configuración global: plugins, usuarios, Credentials (lo usarás en el Nivel 2).
Build History (dentro de un job)	Cada ejecución: #1 , #2 , #3 ... Cada una guarda su log y sus ficheros.

El sitio al que volverás mil veces: dentro de un build, el enlace **Console Output**. Es el log completo, línea a línea, de todo lo que se ejecutó. Cuando algo falle —y va a fallar—, la respuesta está ahí. No adivines: **lee el log**.

NIVEL 1 — Tu primer pipeline

≈ 10 minutos

Objetivo: crear un job, ejecutarlo y entender qué te está contando Jenkins.

Paso 1.1 — Crear el job

1. En el Dashboard, pulsa **+ New Item**.
2. **Enter an item name:** escribe `pipeline-TUNOMBRE` (por ejemplo `pipeline-ana`).
3. Selecciona el tipo **Pipeline** (no "Freestyle project").
4. Pulsa **OK**.

¿Por qué "Pipeline" y no "Freestyle"? Un job *Freestyle* se configura rellenando formularios en la web: es cómodo el primer día e ingobernable el día 100 (no se puede versionar, ni revisar, ni copiar). Un job *Pipeline* se define en un **script** que puede vivir en git. Es lo que se usa hoy en día, y es lo que hace que a esto se le llame **Pipeline as Code**.

Paso 1.2 — Escribir el pipeline

Baja hasta la sección **Pipeline** (al final de la página de configuración). Deja **Definition:** *Pipeline script* y escribe esto en el recuadro:

```

pipeline {
  agent any

  stages {
    stage('Saludo') {
      steps {
        echo "Hola, soy el build número ${env.BUILD_NUMBER}"
        sh 'date'
        sh 'whoami'
        sh 'pwd'
      }
    }
    stage('¿Tengo Docker?') {
      steps {
        sh 'docker version --format "Servidor Docker: {{.Server.Version}}"'
      }
    }
  }
}

```

Pulsa **Save**.

Qué significa cada línea

Línea	Qué hace
<code>pipeline { }</code>	Abre un pipeline declarativo. Todo va dentro.
<code>agent any</code>	Dónde se ejecuta. <code>any</code> = donde haya sitio libre.
<code>stages { }</code>	La lista de bloques de trabajo.
<code>stage('Saludo')</code>	Un bloque con nombre. Aparecerá como una columna en el <i>Stage View</i> .
<code>steps { }</code>	Las acciones concretas de ese bloque.
<code>echo "..."</code>	Escribe un mensaje en el log.
<code>sh 'comando'</code>	Ejecuta un comando de shell de Linux.
<code>\${env.BUILD_NUMBER}</code>	Una variable que Jenkins rellena solo: el número de ejecución.

Paso 1.3 — Lanzarlo y leer el resultado

1. Pulsa **Build Now** (menú izquierdo).
2. En **Build History** aparecerá **#1**. Haz clic encima.
3. Entra en **Console Output** y lee el log de arriba abajo.

✓ Comprueba que entiendes esto:

- El **Stage View** del job muestra una columna por `stage`, con el tiempo que tardó cada una.
- `whoami` devuelve `root`: los comandos corren como root dentro del contenedor de Jenkins.
- `pwd` devuelve algo como `/var/jenkins_home/workspace/pipeline-tunombre`. Eso es tu **workspace**: la carpeta de trabajo de este job.
- La segunda etapa demuestra que Jenkins **puede hablar con Docker**. Ese es el motor que hará todo el trabajo pesado a partir del Nivel 2.

Paso 1.4 — Rompe algo a propósito (importante)

Vuelve a **Configure**, cambia `sh 'date'` por `sh 'fecha'` (que no existe), guarda y lanza otra vez.

- El build sale en **rojo**.
- En el Console Output verás `fecha: not found` y `script returned exit code 127`.
- La etapa '¿Tengo Docker?' **no llega a ejecutarse**.

Regla fundamental: si un comando devuelve un código de salida distinto de **0**, el step `sh` falla, la etapa falla y **el pipeline se detiene ahí**. Recuérdalo: en el Nivel 3 vamos a aprovechar exactamente ese comportamiento para construir un quality gate de seguridad.

Deshaz el cambio (`sh 'date'`) antes de continuar.

NIVEL 2 — Código real y un escáner de seguridad

≈ 15 minutos

Objetivo: descargar el código desde GitLab usando una credencial, y pasarle un analizador de secretos de verdad.

Paso 2.1 — Consigue tu token de GitLab (PAT)

Jenkins necesita permiso para clonar el repositorio. Ese permiso es un **Personal Access Token**. Si ya creaste uno en la sesión anterior, reutilízalo. Si no:

1. Entra en GitLab (`http://localhost:8929`).
2. Arriba a la derecha: **avatar ► Edit profile ► Access Tokens**.
3. **Add new token**. Nombre: `jenkins`. Marca los scopes `read_repository` (y `write_repository` si luego quieres subir cambios).
4. **Create y copia el token** (`glpat-...`). No se vuelve a mostrar.

Paso 2.2 — Guarda el token en Jenkins como credencial

1. **Manage Jenkins ► Credentials**.
2. En la tabla, haz clic en el **store System ► Global credentials (unrestricted)**.
3. **+ Add Credentials**.
4. Rellena:

Campo	Valor
Kind	Username with password
Scope	Global
Username	root
Password	tu token glpat-...
ID	gitlab-cred-TUNOMBRE (este nombre lo usarás en el script)
Description	GitLab — TUNOMBRE

5. Create.

Esto es el corazón del DevSecOps aplicado a ti mismo. El `Jenkinsfile` acaba en `git`: si escribes ahí un token, se lo has regalado a todo el que tenga acceso al repositorio... y queda en el **historial para siempre**, aunque luego lo borres. Los secretos van **siempre** en Credentials. Jenkins además los **enmascara** en el log: verás `****`.

Paso 2.3 — El pipeline que analiza código real

Vuelve a tu job ► **Configure** ► sección **Pipeline**, y sustituye todo el script por este (cambia `TUNOMBRE` por el ID real de tu credencial):

```

pipeline {
    agent any

    stages {
        stage('Checkout') {
            steps {
                git url: 'http://devsecops-gitlab:8929/root/devsecops-sample.git',
                    branch: 'main',
                    credentialsId: 'gitlab-cred-TUNOMBRE'
                sh 'ls -la'
            }
        }

        stage('Buscar secretos') {
            steps {
                sh '''
                    docker run --rm \
                    -v "$WORKSPACE":/src -w /src \
                    ghcr.io/gitleaks/gitleaks:v8.18.4 \
                    detect --source /src \
                        --report-format json \
                        --report-path /src/gitleaks-report.json \
                        --redact --verbose || true
                '''
            }
        }
    }

    post {
        always {
            archiveArtifacts artifacts: '*.json', allowEmptyArchive: true
        }
    }
}

```

Guarda y pulsa **Build Now**.

Las cuatro cosas que tienes que entender de este script

Fragmento	Por qué es así
<code>http://devsecops-gitlab:8929</code>	No es <code>localhost</code> . El job corre en la red de contenedores, y allí GitLab se llama <code>devsecops-gitlab</code> .
<code>credentialsId: 'gitlab-cred-...'</code>	Jenkins busca esa credencial por su ID y la usa. El token no aparece en el script.
<code>docker run ... gitleaks</code>	Jenkins no tiene Gitleaks instalado: lanza un contenedor con la herramienta, la usa, y el contenedor desaparece (<code>--rm</code>).
<code>-v "\$WORKSPACE":/src</code>	Le "presta" tu carpeta de trabajo al contenedor de Gitleaks, que la ve como <code>/src</code> . Sin esto, Gitleaks no tendría nada que analizar.
<code> true</code>	Gitleaks devuelve código 1 cuando encuentra secretos. Sin este truco, el build se pararía ahí. De momento queremos ver el resultado; en el reto final lo quitaremos.
<code>archiveArtifacts</code>	Guarda el informe JSON colgado del build, para poder descargarlo después.

✓ **Resultado esperado:** en el Console Output, Gitleaks reporta **1 hallazgo**: la regla `aws-access-token` en `app.py` , línea 23 (`AWS_SECRET = "AKIAIOSFODNN7FAKEFAKE"`). En la página del build aparece **gitleaks-report.json** en la sección de artefactos: descárgalo y ábrelo.

¿Solo uno? Pero si en `app.py` hay tres secretos. Exacto, y ahí está la lección. Gitleaks funciona con **reglas basadas en patrones conocidos**: sabe qué pinta tiene una clave de AWS (`AKIA...`), un token de GitHub o una clave de Stripe. Pero `DATABASE_PASSWORD = "SuperSecret123!"` no encaja con ningún patrón reconocible: es una cadena cualquiera. **Falso negativo**.

Lo interesante: cuando llegues al Nivel 3 verás que **SonarQube SÍ marca esa contraseña** (*"password detected here, review this potentially hard-coded credential"*) — y en cambio SonarQube no ve el token de AWS. **Cada herramienta tapa el agujero de la otra**. Por eso el pipeline tiene siete etapas y no una.

Nota sobre `--redact` : hace que el log muestre el hallazgo pero **no el valor completo** del secreto. Si tu herramienta de seguridad escupe secretos en claro en un log que luego lee media empresa, has creado un problema nuevo mientras arreglabas el viejo.

NIVEL 3 — El pipeline DevSecOps completo

≈ 15 minutos

Objetivo: dejar de escribir el pipeline en la web y hacer que Jenkins lo **lea del repositorio**. Ese pipeline ejecuta las siete etapas y manda todo a DefectDojo.

Paso 3.1 — El token de SonarQube

1. Entra en SonarQube (`http://localhost:9000`).
2. Arriba a la derecha: **avatar ► My Account ► Security**.

3. En **Generate Tokens**: nombre `jenkins-TUNOMBRE`, tipo **User Token** ► **Generate**.
4. Copia el token (`squ_...`).

Paso 3.2 — La API key de DefectDojo

1. Entra en DefectDojo (`http://localhost:8080`).
2. Menú del usuario (arriba a la derecha) ► **API v2 Key**. Copia la clave (40 caracteres).

Alternativa por consola, si prefieres practicar con la API:

```
curl -s -X POST http://localhost:8080/api/v2/api-token-auth/ \
  --data-urlencode "username=admin" \
  --data-urlencode "password=DevSecOps2024!"
```

Paso 3.3 — Guardar las dos como credenciales *Secret text*

En **Manage Jenkins** ► **Credentials** ► **System** ► **Global credentials** ► **+ Add Credentials**, crea estas dos (tipo **Secret text**):

ID (exacto)	Secret
<code>sonar-token</code>	el <code>squ_...</code> del paso 3.1
<code>defectdojo-api-key</code>	la API key del paso 3.2

Estos dos IDs deben ser exactamente esos, porque así es como los busca el `Jenkinsfile` del repositorio. **Si tu instructor ya las ha creado** (lo verás en la lista de credenciales), **no las dupliques**: sáltate este paso.

Paso 3.4 — Que Jenkins lea el pipeline del repositorio

1. Tu job ► **Configure** ► sección **Pipeline**.
2. Cambia **Definition** de *Pipeline script* a **Pipeline script from SCM**.
3. Rellena:

Campo	Valor
SCM	Git
Repository URL	<code>http://devsecops-gitlab:8929/root/devsecops-sample.git</code>
Credentials	tu <code>gitlab-cred-TUNOMBRE</code>
Branches to build	<code>*/main</code>
Script Path	<code>Jenkinsfile</code>

4. **Save**.

Esto es Pipeline as Code de verdad. Jenkins ya no guarda tu receta: la **lee del repositorio** en cada ejecución. Si alguien cambia el **Jenkinsfile** y hace push, el pipeline cambia solo — y ese cambio ha pasado por un merge request, con revisión, como cualquier otro código. Es la diferencia entre "automatización que alguien configuró un día" y "automatización auditable".

Paso 3.5 — Lanzarlo

1. Pulsa **Build Now**. **Esta primera vez** Jenkins solo está descubriendo qué parámetros tiene el pipeline, así que puede lanzarse con los valores por defecto.
2. A partir de ahora verás el botón **Build with Parameters**. Púlsalo y revisa las opciones:

Parámetro	Qué hace
<code>GIT_URL</code> / <code>GIT_BRANCH</code>	Qué repositorio y rama analizar.
<code>GIT_CRED_ID</code>	Pon aquí tu <code>gitlab-cred-TUNOMBRE</code> .
<code>RUN_SAST</code>	SonarQube. Déjalo marcado: si está apagado, la etapa se omite sola y no rompe nada.
<code>RUN_DAST</code>	OWASP ZAP. Tarda 3-5 min: déjalo desmarcado salvo que te sobre tiempo.
<code>BLOQUEAR_SI_HAY_SECRETOS</code>	El quality gate. De momento, desmarcado.

3. **Build**. Ahora vete al **Console Output** y **quédate mirando** cómo pasa por las etapas.

Paso 3.6 — Qué está pasando en cada etapa

Etapas	Herramienta	Qué busca
1 · Secretos	Gitleaks	Credenciales escritas a mano en el código y en el historial de git.
2 · SAST	SonarQube	Fallos en el código fuente sin ejecutarlo: SQL injection, MD5, command injection...
3 · SCA	Trivy (fs)	CVEs conocidos en las librerías de <code>requirements.txt</code> .
4 · Build	Docker	Construye la imagen de la aplicación. Si no compila, no hay nada que desplegar.
5 · Contenedor	Trivy (image)	CVEs del sistema operativo base de la imagen que acabas de construir.
6 · DAST	OWASP ZAP	Ataca la aplicación en ejecución : cabeceras, XSS, configuraciones inseguras.
7 · Informe	DefectDojo	Sube todos los informes por API y los consolida en un único panel.

✓ **Resultado esperado:** el build acaba en **amarillo (UNSTABLE)**. **Eso es correcto**. Amarillo significa "terminé, pero hay hallazgos de seguridad". La aplicación del laboratorio es vulnerable **a propósito**: si saliera verde, algo estaría mal configurado.

Hasta ahora la seguridad **informaba**. Ahora va a **impedir**.

1. Pulsa **Build with Parameters** y esta vez marca **BLOQUEAR_SI_HAY_SECRETOS**.
2. Lanza el build y observa:
 - El build sale en **rojo (FAILURE)**.
 - Las etapas siguientes **ni siquiera se ejecutan**.
 - En el log aparece: **QUALITY GATE: Gitleaks ha encontrado secretos. Pipeline abortado.**

Este es el fragmento del **Jenkinsfile** que lo hace posible:

```
int rc = sh(returnStatus: true, script: 'docker run ... gitleaks detect ...')

if (rc == 0) {
    echo 'Sin secretos' // SUCCESS
} else if (params.BLOQUEAR_SI_HAY_SECRETOS) {
    error('QUALITY GATE: secretos detectados') // FAILURE – aborta
} else {
    unstable('Secretos detectados (solo aviso)') // UNSTABLE – continúa
}
```

Fíjate en **returnStatus: true**. Sin eso, **sh** abortaría el pipeline automáticamente en cuanto Gitleaks devolviera 1, y **nunca llegarías a decidir qué hacer**. Con **returnStatus** capturas el código de salida y **tú** eliges: informar o bloquear. Es exactamente lo mismo que hacías en GitLab CI con **allow_failure: true|false**.

Cierra el ciclo: arregla la vulnerabilidad

Un control que bloquea solo sirve si alguien arregla lo que señala. Hazlo:

```
# Clona el repo (usa tu PAT)
git clone http://root:TU_PAT@localhost:8929/root/devsecops-sample.git
cd devsecops-sample
git checkout -b fix-secretos # en una rama, para no tocar main

# Edita app.py y saca el secreto del código:
#   AWS_SECRET = "AKIAIOSFODNN7FAKEFAKE"
#   -> AWS_SECRET = os.environ.get("AWS_SECRET")

git commit -am "fix: sacar los secretos del codigo fuente"
git push origin fix-secretos
```

Vuelve a Jenkins, **Build with Parameters**, pon **GIT_BRANCH** = **fix-secretos**, deja **BLOQUEAR_SI_HAY_SECRETOS** marcado, y lanza.

Sigue en ROJO. Y esto es lo más importante que vas a aprender hoy.

Has quitado el secreto del código, pero Gitleaks no analiza solo el código actual: analiza **todo el historial de git**. El commit viejo con el token sigue ahí, y seguirá ahí para siempre. En el log verás algo como `3 commits scanned ... leaks found: 1`, apuntando al **commit original**.

Un secreto que se ha commiteado alguna vez está quemado. Cualquiera que haya clonado el repo lo tiene. Borrarlo del fichero no lo borra de la historia, ni de los *forks*, ni de los *backups*, ni de la caché de GitHub. La única respuesta correcta es **rotar la credencial**: invalidarla y emitir una nueva.

Compruébalo tú mismo. Estos dos comandos analizan **exactamente el mismo código**:

```
# Solo el código actual -> "no leaks found", código de salida 0
docker run --rm -v "$PWD":/src ghcr.io/gitleaks/gitleaks:v8.18.4 \
  detect --source /src --no-git --redact

# El código MÁS el historial -> "leaks found: 1", código de salida 1
docker run --rm -v "$PWD":/src ghcr.io/gitleaks/gitleaks:v8.18.4 \
  detect --source /src --redact
```

Acabas de recorrer el ciclo completo de DevSecOps: detectar → bloquear → corregir → **descubrir que la corrección no bastaba**. Eso último es justamente el motivo de que la seguridad se automatice: para que la herramienta te lo diga en 40 segundos y no un atacante seis meses después.

¿Y cómo se arregla de verdad? Por este orden: **(1) rotar** la credencial expuesta — es lo único imprescindible; **(2)** sacarla del código y llevarla a una variable de entorno o a un gestor de secretos; **(3)** opcionalmente reescribir el historial (`git filter-repo` , BFG), sabiendo que eso obliga a todo el equipo a re-clonar y **no recupera** lo que ya se filtró.

4. Dónde se leen los resultados

Herramienta	Pregunta que responde	Qué mirar
Jenkins :8180	¿Pasó o no pasó?	El color del build, el tiempo de cada etapa en el <i>Stage View</i> , el Console Output y los artefactos descargables. Si activaste el DAST, verás también la pestaña Informe OWASP ZAP .
SonarQube :9000	¿Qué está mal en mi código?	Tu proyecto <code>jenkins-pipeline-TUNOMBRE</code> . Mira sobre todo la pestaña Security Hotspots : contraseña hardcodeda, CSRF desactivado, hash débil, imagen que corre como root... Cada uno con su línea exacta y una explicación. Verás "0 Vulnerabilities" y eso no es un fallo: lee el recuadro de abajo.
DefectDojo :8080	¿Cuál es mi riesgo total?	Producto "DevSecOps Lab" ▶ tu engagement <code>Jenkins · pipeline-tunombre</code> . Todos los hallazgos de todas las herramientas, clasificados por severidad. Cada ejecución añade sus <i>tests</i> al mismo engagement, con el número de build en el título: por eso puedes comparar el "antes" y el "después" de arreglar la vulnerabilidad. En las versiones recientes de DefectDojo el menú Products se llama Assets (y <i>Endpoints</i> pasó a <i>Locations</i>). Si no encuentras "Products", busca "Assets": es lo mismo.

Fíjate en un detalle que lo explica todo. Dentro del producto "DevSecOps Lab" de DefectDojo conviven ahora engagements que vienen de **GitLab CI** (`Pipeline #...`) y de **Jenkins** (`Jenkins · ...`). Dos orquestadores distintos alimentando **un mismo panel de riesgo**. A la empresa no le importa qué CI usó cada equipo: le importa cuántas vulnerabilidades críticas tiene abiertas. Eso es **gestión de vulnerabilidades**.

SonarQube dice "0 Vulnerabilities". ¿Está roto? No. Estás usando la **edición Community** (gratuita), y la detección automática de inyecciones (SQL injection, command injection...) necesita *taint analysis*: seguir el dato contaminado desde que entra por el usuario hasta que llega a la consulta. Eso es una **función de pago** (Developer Edition en adelante).

Lo que sí te da Community son **6 Security Hotspots**: puntos que un humano debe revisar (la contraseña en el código, el hash MD5, el CSRF desactivado, la imagen corriendo como root). **Es un hallazgo del ejercicio, no un error**: las herramientas gratuitas cubren una parte, las de pago otra, y ninguna sola es suficiente. Fíjate en que la `API_KEY` hardcodeda **sí** la cazó Gitleaks, y los CVEs de las librerías **sí** los cazó Trivy. Por eso el pipeline tiene siete etapas y no una.

Ejercicio de análisis (contéstalo con tus resultados)

1. ¿Qué encontró **solo** Gitleaks y ninguna otra herramienta? ¿Por qué?
2. ¿Qué encontró **solo** Trivy? ¿Es un fallo de tu código o de otra persona?

3. SonarQube marca `hashlib.md5(...)` como hotspot de gravedad **baja**. ¿Por qué es un problema de seguridad y no simplemente "código antiguo"? ¿Te parece bien esa gravedad?
4. El SQL injection de `get_user()` **no lo encuentra nadie** en este pipeline. ¿Qué herramienta o edición harían falta? ¿Qué otro control (revisión de código, tests, WAF) lo compensaría mientras tanto?
5. Si tuvieras que **bloquear solo una** de las siete etapas en tu empresa mañana, ¿cuál elegirías y con qué argumento se lo venderías al equipo de desarrollo?

5. Retos extra (si te sobra tiempo)

1. **DAST completo:** lanza el build con `RUN_DAST` marcado y abre la pestaña **Informe OWASP ZAP** del build. ¿Qué cabeceras de seguridad faltan?
2. **Trivy bloqueante:** en el `Jenkinsfile`, añade `--exit-code 1` al comando de Trivy para que los CVEs críticos rompan el build.
3. **Escaneo nocturno:** en **Configure ► Build Triggers ► Build periodically** pon `H 2 * * *`. Ese `H` significa "a una hora repartida": evita que todos los jobs del mundo se lancen a las 2:00 en punto.
4. **Tu propio repositorio:** copia el `Jenkinsfile` a tu repo `mi-app-tunombre` de la sesión anterior, y apunta el job ahí.
5. **Notificación:** añade al bloque `post { failure { ... } }` un `echo` con el `${BUILD_URL}`. En producción, ahí es donde iría el aviso a Slack o correo.

6. Cuando algo falla

Antes de preguntar: abre el **Console Output** y busca la primera línea en rojo. El 90% de los problemas están en esta tabla.

Síntoma	Causa y solución
El build se queda en pending / "waiting for next available executor"	Hay compañeros ejecutando y no quedan carriles libres. Espera . Míralo en <i>Build Executor Status</i> .
<code>Could not find credentials entry with ID '...'</code>	El ID de la credencial no coincide con el del script. Revisa mayúsculas, guiones y espacios. Debe ser idéntico .
<code>Authentication failed</code> al clonar	El PAT es incorrecto, ha caducado, o le falta el scope <code>read_repository</code> . Genera uno nuevo.
<code>Could not resolve host: localhost</code> o <i>connection refused</i>	Usaste <code>localhost</code> dentro del pipeline. Cambia a nombre de contenedor (<code>devsecops-gitlab</code> , <code>devsecops-sonarqube</code> ...).
El escáner dice " 0 hallazgos " en un código que sabes que es vulnerable	Casi siempre el volumen está mal montado y el contenedor analizó una carpeta vacía. Añade <code>sh 'ls -la \$WORKSPACE'</code> y, dentro del <code>docker run</code> , un <code>ls /src</code> . Un falso verde es peor que un rojo.
<code>docker: command not found</code>	Estás en un Jenkins sin el cliente de Docker. Avisa al instructor: falta la imagen personalizada del laboratorio.
<code>permission denied ... /var/run/docker.sock</code>	Jenkins no tiene acceso al socket de Docker. Es configuración del laboratorio: avisa al instructor.

La etapa de SonarQube falla con 401 o 403	El token sonar-token es inválido o caducado. Genera otro en <i>My Account ► Security</i> y actualiza la credencial.
SonarQube no responde → etapa SAST OMITIDA	No es un error. SonarQube está apagado para liberar memoria. La etapa sale en verde y el resto del pipeline continúa. Es una decisión del laboratorio, no de tu código.
DefectDojo responde HTTP 400 al importar	Suele ser el nombre del tipo de escaneo o que falta el producto. El Jenkinsfile ya manda product_type_name para crearlo solo; comprueba la API key.
La etapa de ZAP tarda muchísimo	Es normal: un <i>baseline</i> son 3-5 minutos. Si no tienes tiempo, desmarca RUN_DAST .
El pipeline aborta en Checkout con FATAL: el demonio Docker del HOST no ve \$WORKSPACE	Es una protección deliberada , no un fallo tuyo: el laboratorio está mal montado y los escáneres analizarían una carpeta vacía (saldría "verde" sin haber comprobado nada). Avisa al instructor.
Gitleaks NO pudo ejecutarse (código 125/126/127)	No es un hallazgo de seguridad: es que la imagen no se pudo descargar o un flag está mal escrito. Mira las líneas anteriores del log.

No aparece **Build with Parameters**

Jenkins registra los parámetros la **primera vez** que lee el **Jenkinsfile** . Lanza el job una vez con **Build Now** y vuelve a mirar.

7. Chuleta de referencia

Estructura de un Jenkinsfile

```
pipeline {
  agent any                // dónde se ejecuta
  options { timestamps() } // cómo se comporta el job
  parameters { booleanParam(name: 'X', defaultValue: false) }
  environment { MI_VAR = 'valor' } // variables para todo el pipeline
  stages {
    stage('Nombre') {
      when { expression { return params.X } } // ejecutar solo si...
      steps { sh 'comando' }
    }
  }
  post {                    // siempre al final
    always { archiveArtifacts artifacts: '*.json', allowEmptyArchive: true }
    success { echo 'verde' }
    unstable { echo 'amarillo' }
    failure { echo 'rojo' }
  }
}
```

Steps más usados

Step	Qué hace
<code>sh 'comando'</code>	Ejecuta shell. Si devuelve $\neq 0$, falla la etapa.
<code>sh(returnStatus: true, script: '...')</code>	Ejecuta y te devuelve el código de salida sin fallar.
<code>sh(returnStdout: true, script: '...')</code>	Ejecuta y te devuelve la salida de texto .
<code>echo 'texto'</code>	Escribe en el log.
<code>git url: '...', branch: '...', credentialsId: '...'</code>	Clona un repositorio.
<code>withCredentials([string(credentialsId:'x', variable:'V')]) { }</code>	Inyecta un secreto como variable \$V .
<code>archiveArtifacts artifacts: '*.json'</code>	Guarda ficheros en el build.
<code>error 'mensaje'</code>	Aborta el build en rojo .
<code>unstable 'mensaje'</code>	Marca el build en amarillo y continúa.
<code>catchError(buildResult:'UNSTABLE', stageResult:'FAILURE') { }</code>	Si algo falla dentro, no tumba el pipeline.

Variables de entorno útiles

Variable	Contenido
<code>\$WORKSPACE</code>	Carpeta de trabajo del build.
<code>\$BUILD_NUMBER</code>	Número de ejecución.
<code>\$JOB_NAME</code>	Nombre del job.
<code>\$BUILD_URL</code>	Enlace directo a este build.
<code>\$GIT_COMMIT / \$GIT_BRANCH</code>	Commit y rama analizados.

Equivalencias GitLab CI ↔ Jenkins

GitLab CI	Jenkins	Concepto
<code>.gitlab-ci.yml</code>	<code>Jenkinsfile</code>	La receta, en el repo
<code>stages:</code>	<code>stages { }</code>	Bloques del pipeline
<code>script:</code>	<code>steps { sh '...' }</code>	Los comandos
<code>variables:</code>	<code>environment { }</code>	Variables
Settings ► CI/CD ► Variables	Manage Jenkins ► Credentials	Los secretos
<code>artifacts:</code>	<code>archiveArtifacts</code>	Guardar informes
<code>allow_failure: true</code>	<code>unstable()</code>	Avisar sin bloquear
<code>allow_failure: false</code>	<code>error()</code>	Bloquear (quality gate)
<code>rules: / only:</code>	<code>when { }</code>	Ejecutar condicionalmente
GitLab Runner	Agente / executor	Quién ejecuta el trabajo

Comillas y secretos: el detalle que casi nadie explica

En Groovy, `'...'` (comilla simple) y `"..."` (comilla doble) NO son lo mismo, y con secretos la diferencia importa:

```
# CORRECTO – comillas SIMPLES: la variable la expande el SHELL
sh '''docker run ... -Dsonar.token="${SONAR_TOKEN}" ...'''

# INCORRECTO – comillas DOBLES: Groovy mete el secreto en el texto del script
# ANTES de que Jenkins lo vea, y el enmascarado deja de funcionar.
sh """docker run ... -Dsonar.token=${SONAR_TOKEN} ..."""
```

Si Jenkins te avisa con *"A secret was passed to sh using Groovy String interpolation, which is insecure"*, es exactamente esto: cambia las comillas dobles por simples.

Truco final: en cualquier Jenkins, entra en `<URL-de-Jenkins>/pipeline-syntax`. Es un generador: eliges un step, rellenas un formulario y te da la línea exacta lista para pegar en tu `Jenkinsfile`. Es el mejor amigo de quien empieza.